ME416 — Lab 3: Autonomous Driving

Fall 2026. Work in teams of three. Submit everything by **[Friday 11/13, midnight]**, uploaded to Blackboard as described in Section 7. AI is allowed under the course policy; see Section 9.

Your task is to drive a Limo autonomously through the RASTIC ARENA. You'll be given a starting pose, a goal pose, and intermediate poses to pass through. Plan a Dubins path through them and turn it into a trajectory. Then drive the course twice: first using feedforward alone, and then using feedback from the motion capture system (mocap, or OptiTrack) with an LQR controller. The comparison between those runs is the main point of the lab.

Mocap gives you the Limo's pose, so you don't need to solve a localization problem. Your job is to make the robot follow the planned path. You won't use its camera.

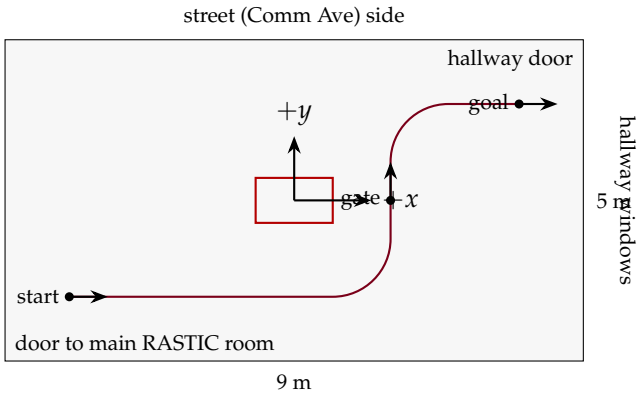
Notation

- Matrices and vectors are bold ($\mathbf{A}$, $\mathbf{K}$); scalars are italic (v, ϕ)
- Vectors are columns, written inline as $[x, y, \theta]^\mathsf{T}$
- $[x, y]^\mathsf{T}$ is a position in the room frame, in meters, and θ is the Limo's heading, measured from $+x$ toward $+y$
- v is the Limo's forward speed and ϕ its steering angle. L is the wheelbase, and r_{min} the smallest turning radius you plan with
- A subscript ref marks the trajectory you planned (x_{ref}, θ_{ref}), and a subscript ff marks the feedforward command (ϕ_{ff})
- ε_y is the cross-track error, the signed distance from the Limo to the path, and $\varepsilon_\theta = \theta - \theta_{ref}$ is the heading error
- Angles in degrees on paper, radians in code, lengths in meters

Use any programming language, but name variables in `camelCase` with a unit suffix when appropriate (e.g. `steerCmd_rad` or `crossTrackErr_m`).

1 The room, the robots, and the wire

The room. The origin is at the center of the red tape rectangle in the ARENA. $+x$ points toward the hallway windows and $+y$ toward the street (Comm Ave). The usable mocap area is approximately 9 m by 5 m, so $[4.5, 2.5]^\mathsf{T}$ is the corner closest to the hallway door and $[-4.5, -2.5]^\mathsf{T}$ is the corner closest to the door into the main RASTIC room.



The robots. Use the Limos labeled "ME416" (a small piece of green tape on the top center of the body). Each team gets one. Its identifier, `limoXXX`, is used in every MQTT topic below. Run it in Ackermann mode.

Driving the Limo. RASTIC runs a bridge on each Limo that turns MQTT messages into motion. Send commands by publishing messages; you do not need to log into the Limo.

- Topic `[limo/limoXXX/cmd]`, message `[{"speed_mps": 0.30, "steer_rad": 0.10}]`
- Publish at [20] Hz. If the bridge receives no command for [300] ms, it stops the Limo. This also stops the car if your program crashes
- Speed is limited to [0.5] m/s by the bridge, and steering to the Limo's mechanical stops

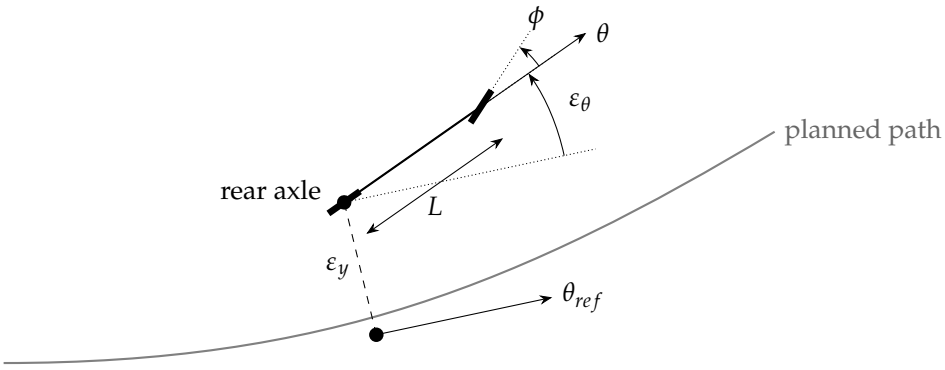
Mocap. Ask RASTIC staff to turn on the system. **Never, not EVER, change any mocap computer settings.** If it's not working, ask RASTIC staff for help.

- Subscribe to the broker at `mqtt://rasticvm.internal`, topic `rb/limoXXX`. You can watch the traffic at `https://mqtt.rastic.org/`
- Each message is JSON: `pos` is a position, `rot` is a quaternion, and `metadata` has the timestamps. For example:

```
{"pos": [-4.7198, 0.0286, 2.6078],
 "rot": [0.0036, -0.719, 0.0116, 0.6949],
 "metadata": {"tracking_error": 0.002243, "frame_number": 11779961, ...}}
```

Notice that one of the three `pos` components represents height, and the quaternion may not use Diebel's component order. You'll work out both in Problem 1.

2 Before you write any code



Problem 1. Work through these questions on paper before coding. You'll use every result.

(a) Set the Limo on the floor pointing at $+x$ and record a mocap message. Then point it at $+y$ and record another. Which component of `pos` is height? How do the other two map onto x and y in the room frame? Which component of `rot` is the scalar? Using Diebel, write the Limo's heading θ in terms of the four components of `rot`, and check it against your two messages

(b) Find the wheelbase L and the maximum steering angle in the AgileX documentation. What turning radius does the bicycle model predict? Now drive the Limo in a full-lock circle under mocap and measure the radius. How do the radii compare? Which will you use for planning?

(c) The Limo is on an arc of radius r, driving at speed v. What steering angle ϕ_{ff} keeps it on that arc? What is ϕ_{ff} on a straight segment?

(d) The Limo is near the planned path, moving at speed v along a segment of curvature $\kappa = 1/r$, and you steer it at $\phi = \phi_{ff} + \delta\phi$. Write the dynamics of $\mathbf{x} = [\varepsilon_y, \varepsilon_\theta]^\mathsf{T}$ linearized about $\varepsilon_y = 0$, $\varepsilon_\theta = 0$, $\delta\phi = 0$, as $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\,\delta\phi$. Does $\mathbf{A}$ or $\mathbf{B}$ change between a straight segment and an arc?

(e) Discretize your $\mathbf{A}$ and $\mathbf{B}$ exactly at the rate you will publish commands. On a straight segment this must reproduce the $\mathbf{A}_d$ and $\mathbf{B}_d$ from class

Problem 2. Record every run, starting when the Limo receives `GO` and ending when it stops. Log at your command rate. One CSV file per run, with this header row, exactly:

```
time_s,xRef_m,yRef_m,thetaRef_rad,x_m,y_m,theta_rad,speedCmd_mps,steerCmd_rad,crossTrackErr_m,headingErr_rad
```

`time_s` starts at zero on `GO`. The `Ref` columns are the reference point your controller used at that step (the projection onto the path), the pose columns are mocap converted to the room frame, and the two error columns are ε_y and ε_θ. Log the errors on the feedforward runs too, even though nothing uses them there. I'll use the same plotting script for every team. Keep the column names and order exactly as specified.

3 Phase 1: Plan, then drive open loop

Objectives:

- Plan a Dubins path through the course poses
- Turn the path into a trajectory
- Drive it with feedforward alone, and measure how far off the Limo ends up

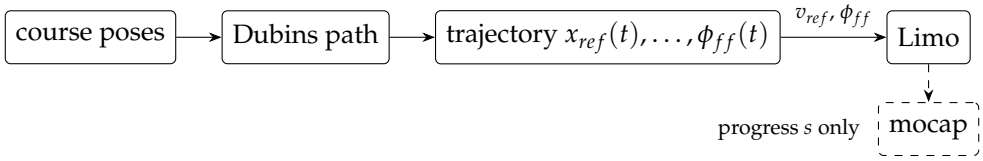
Planning. Use the mocap pose at `GO` as your starting point, then plan through the gates in order to the goal. Between consecutive poses, use the shortest Dubins path with turning radius r_{min}, the

one you chose in Problem 1(b). You may use any Dubins planner you like (the PythonRobotics and MATLAB ones from last year both work), but be prepared to identify and explain the Dubins word selected for each leg.

Trajectory. A path describes the geometry, but not the timing. To create a trajectory, pick v_{max} and a_{max}, build the trapezoidal speed profile over the whole course (one profile, not one per leg), and from it compute $x_{ref}(t)$, $y_{ref}(t)$, $\theta_{ref}(t)$, $v_{ref}(t)$ and $\phi_{ff}(t)$ at your command rate. The speed profile predicts the run time T before the Limo starts. Record that prediction before each run as `predictedTime_s`. You'll compare it with the actual run time in your report.

Progress. Track the path, not the clock. At each step, project the mocap position onto the planned path. The arc length s of that point is your progress, and the reference at that step is the speed from the profile at s and the curvature of the path at s. The run ends when the distance to the goal pose is under a threshold you choose, not when the clock reaches T. Mocap is used for progress on every run, including the feedforward run; what the feedforward run does not do is correct the steering.

Feedforward. Command only $v_{ref}(s)$ and $\phi_{ff}(s)$. Mocap sets your progress along the path and nothing else. You can mark the planned path with tape to make the motion easier to see. Compare the recorded track with the planned path.



Milestone. By the end of the week you should be able to:

(a) Plan a Dubins path through any course and plot it over the room

(b) Run Course 1 open loop and plot the planned path and the mocap track on the same axes, to scale

4 Phase 2: Close the loop

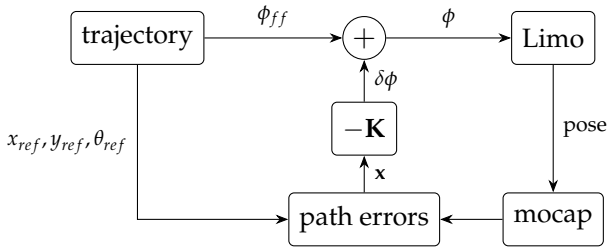
Objectives:

- Compute ε_y and ε_θ from mocap in real time
- Close the loop with LQR, on top of the feedforward
- Drive every course with both controllers and compare them

The controller. At each step, take the same projected reference point as in Phase 1, compute $\mathbf{x} = [\varepsilon_y, \varepsilon_\theta]^\mathsf{T}$, and steer with

$$\phi = \phi_{ff} - \mathbf{K}\mathbf{x}$$

where $\mathbf{K}$ comes from your discretized $\mathbf{A}$ and $\mathbf{B}$ and a $\mathbf{Q}$ and R that you choose. Compute $\mathbf{K}$ with a tool (MATLAB's `dlqr`, or `scipy.linalg.solve_discrete_are`); you do not need to solve the Riccati equation by hand.



Tuning. Expect to adjust $\mathbf{Q}$ and R several times. Record the values you try and how the Limo responds; you'll discuss them in the report.

The race. The LQR runs are timed. A run counts if its RMS cross-track error is under [5] cm, it passes every gate within [15] cm, and it stops within [10] cm and [10°] of the goal pose; a run that fails any of these has no time. Your time on a course is `runTime_s` from the run file you hand in, and the leaderboard on Blackboard lists every team's valid times per course. Go faster by raising v_{max} and a_{max}, and find out what that does to the error.

The courses. Three course files will be posted on Blackboard on [date]. Each is a CSV with one pose per row, `x_m,y_m,theta_deg`: the first row is the start, the last row is the goal, and every row between is a gate the Limo must pass through at that heading. Course 1 is $[-3.5, -1.5, 0°]^\mathsf{T}$ to $[3.5, 1.5, 0°]^\mathsf{T}$ with no gates], so you can begin working with it immediately.

Milestone. By the end of the week you should be able to:

(a) Run every course with feedforward alone and with feedforward + LQR

(b) On Course 1 with feedforward + LQR, keep the RMS cross-track error under [5] cm and stop within [10] cm and [10°] of the goal pose

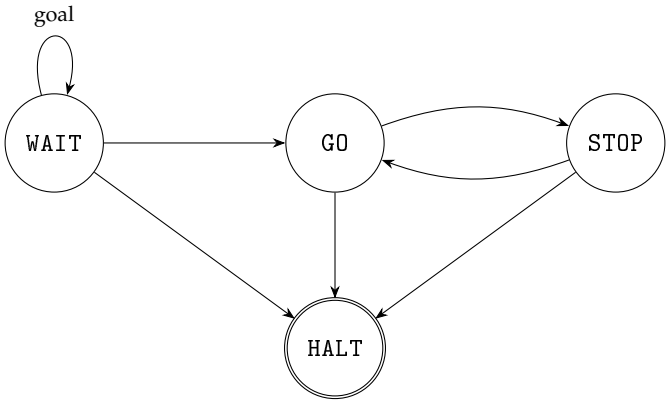
5 Phase 3: Run on command

During the demo, I'll start the runs remotely. Your program must subscribe to two topics for your Limo and respond to their messages.

Run state, on `cmd/limoXXX`, case sensitive:

- `WAIT`: the idle state before the run. Nothing moves
- `GO`: plan from the current pose and drive the course
- `STOP`: the Limo must be stopped within 100 ms. It may resume on the next `GO`, from wherever it stopped
- `HALT`: the Limo must be stopped within 100 ms, and must not move again. The program exits gracefully

Course, on `goal/limoXXX`: a message `{"poses": [[x, y, thetaDeg], ...]}`, same order as the course files. Accept it only in `WAIT`, and ignore it in any other state.



Your own stop conditions. Your program must also stop the Limo if the latest mocap message is more than [250] ms old or the Limo leaves the 9 m by 5 m area. Describe any additional stop conditions in your report and explain why you added them.

You can test these commands by publishing to your team's topics and monitoring traffic at `https://mqtt.rastic.org/`.

Milestone. By the end of the week you should be able to:

(a) Load a course from `goal/limoXXX`, and run it start to finish from a `GO` you didn't send from your own program

(b) Stop on `STOP` and resume on `GO`, and stop for good on `HALT`

6 Rules

- No course pose is hardcoded. Courses come from the files or from `goal/limoXXX`
- The same code drives both controllers. Switching from feedforward to feedforward + LQR is a single setting
- The only feedback term is $-\mathbf{Kx}$, with $\mathbf{K}$ from your own $\mathbf{A}_d$ and $\mathbf{B}_d$. No PID anywhere, on steering or on speed. If a model hands you one, that's the moment to ask it for what you specified instead
- The Limo is placed by hand near the start pose, not on it. The planner deals with the difference
- Remember the 15-minute rule: if you've made no progress after 15 minutes, ask for help

7 Deliverables

One person per team uploads **one zip file** to the Lab 3 group assignment on Blackboard. Use your team number with two digits (`07`, not `7`) wherever it says `NN` below. Use the exact filenames shown below. Missing or incorrectly named files will be treated as not submitted.

```
ME416_Lab3_TeamNN.zip
  ME416_Lab3_TeamNN_report.pdf
  ME416_Lab3_TeamNN_summary.csv
  runs/
    ME416_Lab3_TeamNN_course1_ff.csv      ME416_Lab3_TeamNN_course1_lqr.csv
    ME416_Lab3_TeamNN_course2_ff.csv      ME416_Lab3_TeamNN_course2_lqr.csv
    ME416_Lab3_TeamNN_course3_ff.csv      ME416_Lab3_TeamNN_course3_lqr.csv
  videos/
    ME416_Lab3_TeamNN_course1_ff.mp4
    ME416_Lab3_TeamNN_course1_lqr.mp4
```

```
    ME416_Lab3_TeamNN_course2_lqr.mp4
    ME416_Lab3_TeamNN_course3_lqr.mp4
    ME416_Lab3_TeamNN_runState.mp4
  code/
    README.txt
    ...
  ai/
    ...
```

The runs. Six CSV files, one per course and controller, in the format of Problem 2. Each CSV must cover one continuous run and match the corresponding video, where required.

The summary. One CSV, one row per run (six rows), with this header row, exactly:

```
course,controller,rmsCrossTrack_m,maxCrossTrack_m,finalPosErr_m,finalHeadingErr_deg,predictedTime_s,runTime_s
```

`course` is `1`, `2` or `3`, and `controller` is `ff` or `lqr`. Compute these values from the run files rather than entering them manually; `predictedTime_s` comes from the speed profile, before the run.

The videos. Record each run in one continuous take, keeping the entire Limo in frame. You may want to lay out the path with tape or show a plot of it next to the Limo. The `runState` video shows a run being started with `GO`, stopped with `STOP`, resumed with `GO`, and ended with `HALT`, with the published messages visible on screen. If the zip is over Blackboard's size limit, leave out the `videos/` folder and put a `videoLinks.txt` in its place, with one line per video: the file name above, a space, and a link that works for anyone at BU.

The code (and not a picture of the code).

- I must be able to run your code. `README.txt` says what to install and gives the one command that runs a course, with the setting that picks the controller
- Your code should be well-structured, documented, and commented
  - Well-structured code has no unfinished placeholder functions or large commented-out blocks from abandoned attempts
  - Well-documented means explaining your algorithms clearly enough that a reader can understand and evaluate them. References to papers are fine when appropriate. Don't make the reader reconstruct the mathematics from the code
  - Well-commented means explaining decisions that aren't obvious from the code. For any hardcoded parameter, such as `kp = 1.23`, explain how you chose it and how sensitive the results are to its value. Some systems demand extraordinary precision; others work across a broad range. Your $\mathbf{Q}$ and R are examples of parameters that need justification

The conversations. `ai/` holds the exported conversations with every model you used for this lab.

The report, four pages at most, plus unlimited appendices. Page 1 starts with this block, filled in:

```
Team: NN        Limo: limoXXX
Members: Last, First; Last, First; Last, First
Milestones met: Phase 1 [a, b]   Phase 2 [a, b]   Phase 3 [a, b]
```

Then answer the following questions concisely, but with enough detail to support your conclusions:

(a) Your answers to Problem 1, including the measured and predicted turning radius

(b) For each course, the planned path, the feedforward track and the LQR track on one plot of the room, to scale, from your run files

(c) Where the feedforward run drifts, and why. Which part of the model is wrong, and how can you tell from the data?

(d) The $\mathbf{Q}$ and R you ended with, the ones you tried before, and what each one did to the Limo. What happens to the steering when R gets small?

(e) Your summary table. Is LQR better everywhere, or are there places it isn't? Where did the run time differ from the time you predicted, and what in the model accounts for the gap?

(f) What worked, and what didn't? Explain your design choices, the problems you encountered, and how you resolved them. Think of this section as guidance you would have wanted when starting the lab

(g) A contribution statement: who did what, what a model wrote, what you wrote, and what you changed in what the model wrote

I may ask your team to discuss its work in person. Your lab code may also appear on an exam, so make sure you understand how it works.

8 Sidequest

For an extra challenge, pair up with another team and run two Limos on intersecting courses at the same time [courses to be posted]. Subscribe to the other Limo's `rb/limoXXX` to avoid the other robot, then return to your planned trajectory. A collision means contact with your Limo's front bumper. Driving backward into another robot does not count as avoiding it. Each team should submit a video showing both robots and one page explaining how it predicted the other Limo's motion.

9 AI

You may use any AI model to help write your lab code. You must still understand every line, explain why it's there, and follow the naming convention.